

Practice Problems

Try to solve these practice problems

Problem 1: Implement a Finite Discrete Signal

Implement a class named `DiscreteSignal` that represents a finite discrete-time signal over an inclusive integer time range.

Your class must support the following methods:

- `DiscreteSignal(start_time, end_time)`
 - Creates a zero-valued signal over the given time range.
- `set_value_at_time(t, value)`
 - Sets the signal value at time `t`.
- `get_value_at_time(t)`
 - Returns the signal value at time `t`.
 - If `t` lies outside the stored range, return 0.
- `shift(k)`
 - Returns a new signal representing $x[n - k]$.
 - A positive value of `k` shifts the signal to the right, while a negative value shifts it to the left.

Example

Suppose the signal is

$$\begin{array}{c|ccc} n & -1 & 0 & 1 \\ \hline x[n] & 2 & 1 & 3 \end{array}$$

After calling

`shift(2)`

the resulting signal becomes

$$\begin{array}{c|ccc} n & 1 & 2 & 3 \\ \hline x[n] & 2 & 1 & 3 \end{array}$$

Problem 2: Compute the Output of an LTI System

Using your `DiscreteSignal` class, implement a class named `LTISystem` that stores a finite impulse response $h[n]$.

Implement the following method:

- `output(input_signal)`
 - Computes the output signal using the discrete-time convolution formula

$$y[n] = \sum_{k=-\infty}^{\infty} x[k] h[n - k].$$

Your implementation should satisfy the following requirements:

- Automatically determine the output time range.
- Compute every output sample using the convolution formula.
- Do **not** use built-in convolution functions such as `numpy.convolve()`, `scipy.signal.convolve()`, or similar functions.

Example

Input signal

$$\begin{array}{c|ccc} n & 0 & 1 & 2 \\ \hline x[n] & 1 & 2 & 1 \end{array}$$

Impulse response

$$\begin{array}{c|cc} n & 0 & 1 \\ \hline h[n] & 1 & -1 \end{array}$$

Your program should print:

- Output time range
- Output signal values